Here is the entire walkthrough and architectural guide combined into a single, comprehensive `.md` document.

You can save this directly as `mcp_weather_complete_guide.md` (or run `python3 -c "import sys; ..."` / paste it into any editor):

```
# Open-Meteo MCP Weather Server & Local LLM Integration Guide

This document combines the technical design, operational mechanics, and
integration architecture for the Open-Meteo Model Context Protocol (MCP)
server and the local LLM orchestration bridge.

---

## Part 1: Open-Meteo MCP Weather Server (`weather_server.py`)

### 1. Overview
The `weather_server.py` script functions as an MCP server. It acts as a
standardized capability provider: it exposes discrete, well-typed weather
tools that an AI agent (such as Claude Desktop or a local LLM) can discover
and run via standard input/output (`stdio`).

The server targets the Open-Meteo API (`https://api.open-
meteo.com/v1/forecast`), an open-access endpoint requiring no API tokens,
accounts, or rate-limit configurations.

### 2. Architecture & Data Flow

```text
+----------------------+      JSON-RPC (stdio)      +-------------------+
| Host (Claude / Script)| <========================> | weather_server.py |
+----------------------+                            +-------------------+
                                                              |
                                                    HTTP GET
(httpx2/httpx)
                                                              v
                                                    +-------------------+
                                                    |  Open-Meteo API   |
                                                    +-------------------+
```

- **Communication Medium**: JSON-RPC messages transferred across `stdin` and `stdout`.
- **Asynchronous Networking**: Uses an async HTTP client to ensure non-blocking concurrent request handling.
- **Failure Boundaries**: HTTP status checks (`raise_for_status`) and guarded dictionary lookups (`data.get(...) or {}`) prevent unhandled crashes on invalid coordinates or malformed API responses.

3. Tool Specifications
```

A. `get_current_weather(latitude: float, longitude: float) -> str`

- **Purpose**: Fetches live current weather conditions.
- **Parameters**: `latitude` and `longitude` coordinates.
- **Mechanism**:
- Calls Open-Meteo with `current="temperature_2m,relative_humidity_2m,weather_code"` and `temperature_unit="fahrenheit"`.
- Extracts the `"current"` block safely.
- Formats temperature, humidity, and the numeric WMO (World Meteorological Organization) weather code.

B. `get_tomorrow_weather(latitude: float, longitude: float) -> str`

- **Purpose**: Retrieves next-day temperature extremes and precipitation odds.
- **Parameters**: `latitude` and `longitude` coordinates.
- **Mechanism**:
- Requests `daily="temperature_2m_max,temperature_2m_min,precipitation_probability_max"`, `forecast_days=2`, and `"timezone": "auto"`.
- **Critical Design Detail**: `"timezone": "auto"` ensures daily boundaries align with the target location's solar calendar day rather than rolling over early to UTC.
- Reads index `1` (`index 0` = today, `index 1` = tomorrow).

C. `get_weekly_weather(latitude: float, longitude: float) -> str`

- **Purpose**: Generates a 7-day weather forecast summary.
- **Parameters**: `latitude` and `longitude` coordinates.
- **Mechanism**:
- Pulls `daily="temperature_2m_max,temperature_2m_min"`.
- Safely iterates through `time`, `max_temps`, and `min_temps` arrays using index-safe clipping (`range(min(...))`).
- Emits an itemized daily forecast list.

Part 2: Local LLM MCP Bridge Client (`ollama_weather_client.py`)

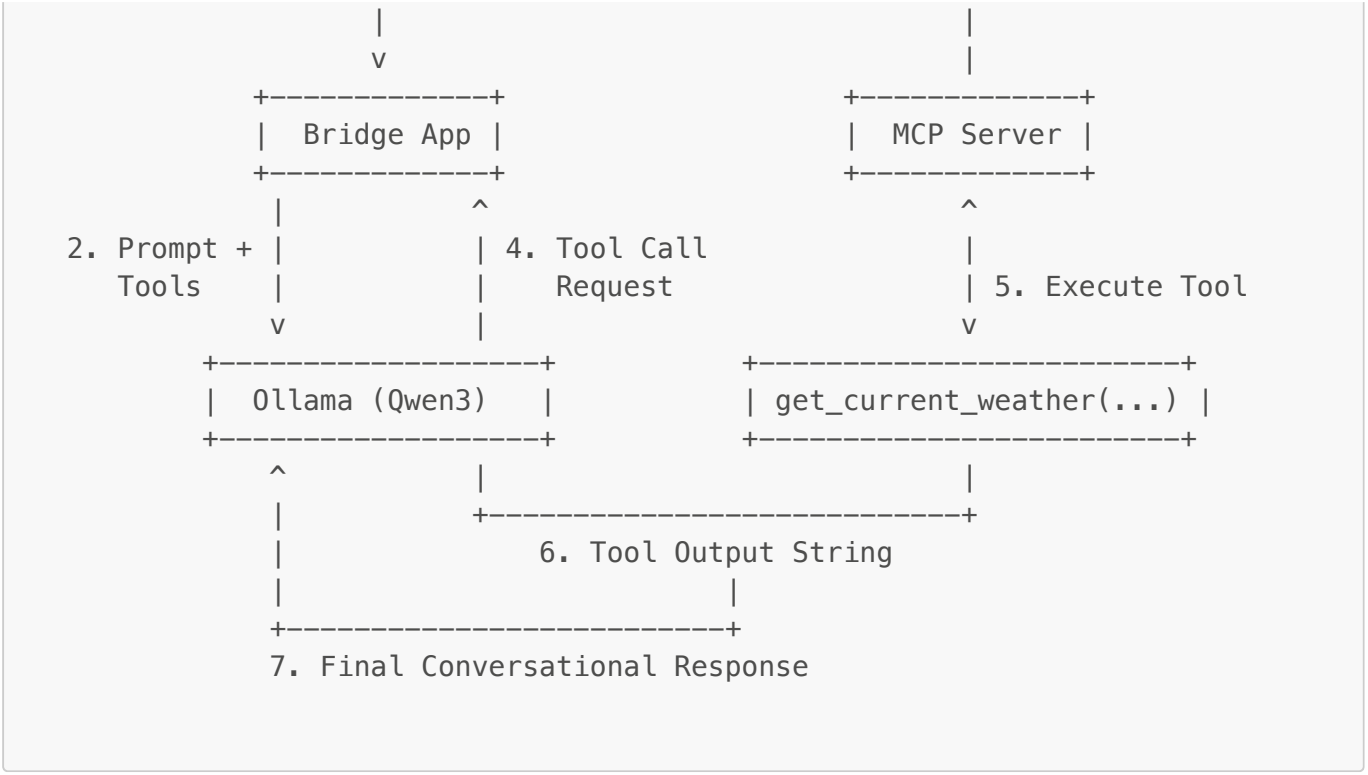
1. Overview

Ollama is purely an inference engine—it runs model weights and produces tokens, but it lacks native support for managing MCP servers, `.mcpb` bundles, or background child processes.

The `ollama_weather_client.py` script functions as the **MCP Host & Orchestrator**. It bridges the local model (`qwen3:4b`, `llama3.1`, etc.) with `weather_server.py`.

2. The Execution Flow

```
                    1. Spawn Subprocess & Handshake
                +-----------------------------------+
```



3. Step-by-Step Breakdown

1. **Subprocess Initialization**: The bridge starts `weather_server.py` via `stdio_client()` and initializes the JSON-RPC session.
2. **Dynamic Schema Discovery**:

- Calls `session.list_tools()` to inspect what the server provides.
- Maps MCP tool definitions (`tool.name`, `tool.description`, `tool.input_schema`) into Ollama's OpenAI-compatible JSON function schema.
- *Benefit*: Any tool added or removed in `weather_server.py` is immediately recognized without updating client code.

3. **Geographic Reasoning & Tool Call Decision (Pass 1)**:

- The user enters a natural query (e.g., `"weather in Paris"`).
- The model uses its internal knowledge to look up Paris coordinates (`48.8584, 2.3200`).
- It outputs a tool-call request matching `get_current_weather` with the inferred parameters.

4. **Execution & Feedback Loop (Pass 2)**:

- The bridge intercepts the tool call, sends it over the stdio session, and collects the returned string from Open-Meteo.
- The returned text is appended to the message history with `role: "tool"`.
- Ollama runs a second pass over the history, translates the numeric weather code into readable terms, and provides the final answer.

Part 3: Complete Reference Source Code

A. `weather_server.py`

```
import httpx2
from mcp.server.mcpserver import MCPServer

app = MCPServer("Open-Meteo Weather Server")
BASE_URL = "[https://api.open-meteo.com/v1/forecast](https://api.open-
meteo.com/v1/forecast)"


@app.tool()
async def get_current_weather(latitude: float, longitude: float) -> str:
    """Get the absolute live, current weather for a specific latitude and
longitude."""
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "current": "temperature_2m,relative_humidity_2m,weather_code",
        "temperature_unit": "fahrenheit",
    }
    try:
        async with httpx2.AsyncClient(timeout=10.0) as client:
            response = await client.get(BASE_URL, params=params)
            response.raise_for_status()
            data = response.json()
    except Exception as e:
        return f"Failed to retrieve current weather: {e}"

    curr = data.get("current") or {}
    if not curr:
        return "Current weather conditions unavailable for this location."

    temp = curr.get("temperature_2m", "N/A")
    humidity = curr.get("relative_humidity_2m", "N/A")
    code = curr.get("weather_code", "N/A")

    return (
        f"Current Conditions:\n"
        f"Temperature: {temp}°F\n"
        f"Humidity: {humidity}%\n"
        f"WMO Weather Code: {code}"
    )


@app.tool()
async def get_tomorrow_weather(latitude: float, longitude: float) -> str:
    """Get tomorrow's weather forecast including max/min temperatures."""
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "daily":
"temperature_2m_max,temperature_2m_min,precipitation_probability_max",
        "temperature_unit": "fahrenheit",
        "forecast_days": 2,
```

```
        "timezone": "auto",
    }
    try:
        async with httpx2.AsyncClient(timeout=10.0) as client:
            response = await client.get(BASE_URL, params=params)
            response.raise_for_status()
            data = response.json()
    except Exception as e:
        return f"Failed to retrieve tomorrow's forecast: {e}"

    daily = data.get("daily") or {}
    temps_max = daily.get("temperature_2m_max") or []
    temps_min = daily.get("temperature_2m_min") or []
    precip_prob = daily.get("precipitation_probability_max") or []

    if len(temps_max) < 2 or len(temps_min) < 2:
        return "Weather data unavailable for tomorrow."

    return (
        f"Tomorrow's Forecast:\n"
        f"High: {temps_max[1]}°F\n"
        f"Low: {temps_min[1]}°F\n"
        f"Max Rain/Snow Probability: {precip_prob[1] if len(precip_prob) >
1 else 'N/A'}%"
    )


@app.tool()
async def get_weekly_weather(latitude: float, longitude: float) -> str:
    """Get a summary of the 7-day weather forecast."""
    params = {
        "latitude": latitude,
        "longitude": longitude,
        "daily": "temperature_2m_max,temperature_2m_min",
        "temperature_unit": "fahrenheit",
        "timezone": "auto",
    }
    try:
        async with httpx2.AsyncClient(timeout=10.0) as client:
            response = await client.get(BASE_URL, params=params)
            response.raise_for_status()
            data = response.json()
    except Exception as e:
        return f"Failed to retrieve weekly forecast: {e}"

    daily = data.get("daily") or {}
    dates = daily.get("time") or []
    max_temps = daily.get("temperature_2m_max") or []
    min_temps = daily.get("temperature_2m_min") or []

    if not dates:
        return "7-day weather outlook unavailable."

    summary = "7-Day Weather Outlook:\n"
```

```
    for i in range(min(len(dates), len(max_temps), len(min_temps))):
        summary += (
            f"- {dates[i]}: High {max_temps[i]}°F / Low {min_temps[i]}°F\n"
        )

    return summary


if __name__ == "__main__":
    app.run()
```

B. ollama_weather_client.py

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
import ollama

SERVER_PATH = "/ABSOLUTE/PATH/TO/server/main.py"
MODEL_NAME = "qwen3:4b"


async def main():
    server_params = StdioServerParameters(
        command="python3",
        args=[SERVER_PATH],
        env=None,
    )

    print("Connecting to MCP Weather Server...")
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()

            tool_list = await session.list_tools()
            ollama_tools = [
                {
                    "type": "function",
                    "function": {
                        "name": tool.name,
                        "description": tool.description,
                        "parameters": getattr(
                            tool,
                            "input_schema",
                            getattr(tool, "inputSchema", {}),
                        ),
                    },
                }
                for tool in tool_list.tools
```

```
            ]

            print(
                f"Connected. Registered tools: {[t.name for t in
tool_list.tools]}"
            )
            print(f"Loaded Model: {MODEL_NAME}")
            print("-" * 50)

            messages = [
                {
                    "role": "system",
                    "content": (
                        "You are an assistant with access to weather tools.
"
                        "When the user asks about the weather for a city or
region, "
                        "use your geographic knowledge to estimate latitude
and longitude, "
                        "then invoke the appropriate weather tool. Always
use the returned tool data."
                    ),
                }
            ]

            while True:
                user_input = input("\nYou: ").strip()
                if not user_input or user_input.lower() in ("exit",
"quit"):
                    break

                messages.append({"role": "user", "content": user_input})

                response = ollama.chat(
                    model=MODEL_NAME, messages=messages, tools=ollama_tools
                )

                if (
                    hasattr(response.message, "thinking")
                    and response.message.thinking
                ):
                    print(f"\n[Thinking]:
{response.message.thinking.strip()}")

                if response.message.tool_calls:
                    messages.append(response.message)

                    for call in response.message.tool_calls:
                        fn_name = call.function.name
                        fn_args = call.function.arguments

                        print(f"-> [MCP Tool Call] {fn_name}({fn_args})")

                        result = await session.call_tool(
```

```
                            name=fn_name, arguments=fn_args
                        )
                        result_text = "\n".join(
                            c.text for c in result.content if hasattr(c,
"text")
                        )

                        messages.append({
                            "role": "tool",
                            "name": fn_name,
                            "content": result_text,
                        })

                    final_response = ollama.chat(
                        model=MODEL_NAME, messages=messages
                    )
                    messages.append(final_response.message)
                    print(f"\nQwen: {final_response.message.content}")
                else:
                    messages.append(response.message)
                    print(f"\nQwen: {response.message.content}")


if __name__ == "__main__":
    asyncio.run(main())
```